

Self-Evolving RTO / COD Fraud Risk Engine

Design Document — Razorpay AI Buildathon 2026, Track 2: AI Risk Manager

Track	02 — AI Risk Manager (Return-risk scorer)
Loss class targeted	COD Return-to-Origin (RTO) risk
Submission deadline	September 5, 2026

1. Problem Statement

India's e-commerce RTO (Return to Origin) rate averages 25-35%, roughly triple the global benchmark of 8-12%. COD orders specifically see RTO rates near 26%, versus under 2% for prepaid orders. Indian sellers collectively lose an estimated Rs 2 lakh crore annually to this problem, and 40-60% of RTO losses trace back to a small, identifiable group of repeat offenders rather than random noise — meaning the pattern is learnable.

Razorpay's own Thirdwatch product explicitly frames the open problem as adaptation, not initial detection: their ML models are built to flag anomalies that **static, rule-based systems would miss** as fraud patterns evolve. Razorpay also operates a live product, RTO Shield, that scores COD orders for risk before dispatch. This confirms the problem is real, current, and already a company priority — not a manufactured hackathon topic.

What we are actually solving: not "can we detect RTO fraud" (largely solved at a basic level with static rules), but "can we build a system that detects when its own detection rules are going stale, and re-derives better ones without a human data scientist manually noticing and rebuilding them." That adaptation loop is the genuine, still-open problem.

1.1 What counts as an RTO/COD 'loss event' here

- **Genuine logistics failure** — real customer, real intent, address or courier issue. Not fraud; should not be over-flagged.
- **Non-serious / impulse COD orders** — no upfront cost to the customer, so low-intent orders get placed and refused at the door. The majority of the RTO rate.
- **Deliberate abuse** — repeat offenders, multi-account/shared-device abuse, promo stacking, high-value orders with no intent to accept.

The system's job: given an order before dispatch, estimate the probability it becomes an RTO / abusive loss, and route it (ship / verify / reject) accordingly.

2. Track 2 Alignment — 'The Bar', Verbatim

Razorpay's official Buildathon page states the bar for Track 2 as:

"Build a working detector, verifier or auto-responder for one class of loss, with measured precision and recall on a held-out test set." "Honest metrics including false-positive cost. Strictly defense-only: anything offense-capable is disqualified."

Every architectural decision below is traced back to one of these four clauses:

Clause	Design response
One class of loss	Scope locked to COD/RTO risk only. No generalization to card fraud, chargebacks, etc.
Held-out test set	Strict train / validation / held-out-test split. Test set touched exactly once, after the model is frozen.
Honest metrics incl. false-positive cost	Cost-weighted fitness function and a reported Rs-value tradeoff (missed-RTO cost vs. wrongly-blocked-genuine-customer cost), not F1 alone.

Strictly defense-only	A dedicated audit gate before deployment confirms no component outputs an exploitable 'how to evade detection' artifact.
-----------------------	--

3. System Architecture — End to End

The system has two operating phases that never run at the same time: an **offline evolution phase** (proposes and tests new detection logic) and an **online deployment phase** (scores real orders using the last frozen, validated version). A detection change only moves from offline to online after it survives a regression check on validation data — it never goes live directly.

3.1 Full component flow

OFFLINE (Python + LLM API)

Generator Agent (LLM)

- > proposes new RTO/COD hypotheses AND candidate engineered features
- > Evaluator (pandas + scikit-learn)
 - > scores each hypothesis/feature on the VALIDATION split only
- > Reflector Agent (LLM)
 - > given concrete false-positive / false-negative examples, diagnoses why a hypothesis failed, proposes a specific mutation
- > Selector
 - > keeps top-k by COST-WEIGHTED fitness, prunes hypotheses unused or unselected for N consecutive rounds (archived, not deleted)
- > Knowledge Graph Linker (networkx)
 - > links hypotheses that overlap, contradict, refine, or merge
- > Notepad (JSON / SQLite)
 - > persists hypothesis text, rule code, lineage, fitness history
- > loops back to Generator for next round

GATE 1: Regression Suite -> re-test new generation on VALIDATION set

- > FAIL -> back to Generator (mutate further)
- > PASS -> freeze ensemble as new version

GATE 2: HELD-OUT TEST SET (touched exactly once)

- > precision, recall, false-positive cost reported honestly

GATE 3: Defense-only Audit

- > confirm no output is an exploitable "how to evade detection" artifact

ONLINE (FastAPI serving, no LLM call at inference)

- > incoming order -> frozen rule ensemble scores it in milliseconds
- > low-confidence verdicts -> Human Review Queue
- > every prediction logged: input, score, verdict, eventual outcome
- > Drift Detector (rolling window: data drift + performance drift, needs outcomes)
 - > threshold breached -> triggers new Generator round (offline)
 - > healthy -> continues serving
- > Spike Monitor (rolling flag-rate, z-score/CUSUM, no outcomes needed)
 - > anomalous spike -> alert + targeted Generator round on spike cluster
 - > normal -> continues serving
- > Promotion check: new version vs. previous on live metrics
 - > regressed -> automatic Rollback to last known-good version
 - > stable/improved -> continues serving new version

3.2 Why two separate phases, not one continuous loop

Evolution (LLM calls, iterative testing) takes seconds-to-minutes per round; a production fraud score has to return in milliseconds. Coupling them would make 'self-evolving' and 'real-time' contradictory. Separating them means: evolution runs periodically or when triggered by drift; deployment always serves a frozen, fast, already-validated version.

4. Components, In Detail

4.1 Generator Agent (LLM)

Given the current Notepad history (what has worked, what hasn't, and why), proposes new candidate hypotheses as executable rule code, and/or new candidate engineered features (e.g. 'device_id reuse count in last 24h', 'account_age_hours bucketed', 'pincode RTO rate, recency-decayed'). Round 1 is cold-started from general domain knowledge of RTO fraud patterns; later rounds are informed by what the Reflector has learned.

Isolation from the data-generation logic (guards against circularity): the Generator is given only the column schema, column descriptions, and domain framing ("this is COD/RTO order data") — never the synthetic generator script, the logistic-model weights, or the phase/drift labels. It has no access to the code that produced the labels it is trying to predict. This is necessary but not sufficient on its own: see Section 5.4 for the decoy-feature check used to actively test whether the Generator is finding real structure or just reconstructing the known generation logic.

Code safety and failure handling: generated rule code runs inside a restricted sandbox (no filesystem, network, or import access beyond a small allow-list; a hard per-hypothesis timeout) rather than raw `eval()`. If a hypothesis errors out or times out, it gets one repair attempt — the Generator is shown the exact exception and asked to fix the specific issue. A hypothesis that fails twice is marked dead-on-arrival and excluded from that round's Evaluator pass rather than crashing the loop. Given the system runs unattended across many rounds before submission, this is treated as an expected failure mode to design for, not an edge case to hope doesn't happen.

4.2 Evaluator (pandas + scikit-learn)

Ground-truth, non-LLM component. Runs every hypothesis / feature against the **validation split** and computes real precision, recall, and F1. Also computes a **cost-weighted score** using an explicit, stated formula on both sides rather than a number on one side and a hand-wave on the other:

- **True positive value** — Rs 150-300 avoided per correctly-flagged RTO (calibrated against published industry figures, stated as an assumption, not asserted as fact).
- **False positive cost** — modeled per-order as that order's own value x an assumed margin (e.g. `order_value x 0.15`), not a single flat average AOV. This matters specifically because Section 1.1 names high-value, no-intent-to-accept orders as one of the three loss types being targeted — a flat average AOV would underprice the FP cost exactly on the order category the system is most likely to flag and most likely to get wrong, which is an easy inconsistency for a panelist to catch. Using the order's own value keeps the cost model consistent with the threat model it's meant to price.

Fitness ranks hypotheses by net Rs impact using these two explicit terms, not raw F1 — this is what makes the false-positive-cost requirement structural rather than a number reported after the fact, and keeps both sides of

the tradeoff equally defensible under questioning.

4.3 Reflector Agent (LLM)

Takes the Evaluator's output plus concrete misclassified examples (specific orders, not summary stats) and diagnoses *why* a hypothesis failed — e.g. 'this rule over-triggers on high-value but legitimate weekend orders' — then proposes a specific, targeted mutation. This is the 'analyze the pattern and correct itself' step: it is shown its own mistakes and reasons about them before the next round, the same way a person adjusts a workout after noticing a specific exercise isn't producing the expected result.

4.4 Selector

Applies evolutionary pressure: keeps the top-k hypotheses by cost-weighted fitness each round, and **prunes** hypotheses that have gone unused or unselected for N consecutive rounds. Pruned hypotheses are archived (kept in the Notepad's history for lineage) rather than deleted, so the active population stays bounded even as total historical hypotheses grow — this directly answers the 'what happens as hypotheses multiply' question.

4.5 Knowledge Graph Linker (networkx)

Represents hypotheses as nodes and their relationships as typed edges, rather than treating them as independent competing rules:

- **overlaps_with** — two hypotheses flag similar sets of orders
- **contradicts** — two hypotheses disagree on the same orders (signals model uncertainty worth investigating)
- **refines** — a mutated child hypothesis, pointing back to its parent
- **merged_into** — the Selector combined two hypotheses into a composite
- **superseded_by** — an older hypothesis a newer one replaced after a drift event

This graph is the single most demo-visible artifact: it makes 'self-evolving' visually verifiable rather than a claim the judges have to take on faith.

4.6 Notepad (JSON / SQLite)

Persistent memory: hypothesis text, executable rule code, parent lineage, iteration born, full fitness history, and status (alive / mutated / merged / pruned / dead). Every component reads from and writes to this — it is the system's record of its own history, which is what allows the Generator to be 'informed' rather than guessing fresh every round.

4.7 Frozen Static Baseline (v1, never re-evolved) — the core thesis proof

This comparison matters more than the LightGBM one, and is added because without it, the drift-recovery demo is a claim rather than a proof. The entire pitch rests on one sentence: "static, rule-based systems miss evolving patterns." That claim needs to be shown, not just asserted.

Method: freeze the rule ensemble produced after evolution on **pre-drift training data only** (v1 — never re-evolved, never shown the drift pattern). Run this frozen v1 against the **post-drift held-out test set** and report its precision/recall/false-positive-cost there. Then run the fully evolved, drift-adapted ensemble (which re-evolved after the Drift Detector triggered) against the *same* post-drift test set.

Frozen v1 (never re-evolved)	->	post-drift test Recall: X%	Precision: Y%
Evolved (drift-adapted)	->	post-drift test Recall: X'%	Precision: Y'%

The gap between these two numbers, on the same test set, is the literal, on-screen proof of the thesis: static rules degrade after drift, the self-evolving system recovers. This is a stronger, more central artifact for the video than the LightGBM comparison and should be prioritized first if time is tight — the LightGBM baseline is a secondary, 'we considered the alternative' data point; this one is the actual argument for why the project exists.

Known confound, stated openly: if the evolved version simply ran more total rounds than frozen v1, an improvement could come from 'more rounds of optimization' rather than specifically adapting to the drift pattern. The rigorous fix is a rounds-matched control — freeze v1 after N rounds on pre-drift data, and separately let a shadow copy run N *additional* rounds on pre-drift data only (same round budget, no drift exposure) — isolating round-count from drift-exposure as the explanation. This is likely more rigor than the buildathon strictly requires and is treated as optional given the timeline, but the one-sentence answer is prepared regardless: 'we controlled for this by [building the shadow control / noting it as a known limitation not yet controlled for]' — decide which before recording the video, don't leave it to be discovered live.

4.8 LightGBM Baseline (secondary comparison, standalone, not part of the evolution loop)

A single gradient-boosted tree model, trained once (not repeatedly) on the full synthetic dataset using all raw features simultaneously, purely as an honest external benchmark. It is evaluated on the same held-out test set as the evolved rule ensemble, and both numbers are reported side by side:

Evolved rule ensemble	->	held-out Recall: X%	Precision: Y%
LightGBM baseline	->	held-out Recall: X'%	Precision: Y'%

This directly pre-empts the 'why not just train a classifier' question: the comparison is shown, not assumed. The tradeoff argued is interpretability and self-correction without retraining infrastructure, versus a small raw-accuracy edge from the trained model. Training this baseline takes seconds to low minutes on CPU (LightGBM on tabular data of this size is not a 'heavy' model — that concern applies to deep learning / LLM-scale training, not gradient-boosted trees here).

5. Data Strategy

5.1 Why synthetic data

No public RTO/COD-fraud dataset exists (confirmed by search — Razorpay does not publish one, and Thirdwatch/Vulcan's training data is proprietary). The synthetic dataset is generated with distributions calibrated against published industry figures: overall COD RTO rate 25-40%, prepaid RTO rate under 2%, Rs 150-300 average loss per RTO order, and 40-60% of RTO loss concentrated in a small repeat-offender segment. Records are fabricated; the underlying distribution is realistic and this is stated plainly in the demo rather than oversold.

5.2 Split methodology (non-negotiable for the track's bar)

Split	Used by	Touched
Train	Generator / Reflector reasoning context	Every round
Validation	Evaluator fitness scoring, Regression Suite	Every round
Held-out Test	Final precision / recall / false-positive-cost report	Exactly once, after freezing

The test set is never used to select, tune, or influence any hypothesis, feature, or model — it is scored exactly once, at the very end, to produce the numbers reported in the video and README. This is what makes the reported precision/recall defensible rather than circular.

5.3 Injected drift scenario

To demonstrate the self-evolving claim concretely (not just describe it), the synthetic data includes a deliberate distribution shift partway through — e.g. a new promo-stacking abuse pattern that early-round hypotheses do not catch. This lets the demo show: detection performance drops -> Drift Detector flags it -> new Generator round triggered -> new hypothesis proposed and validated -> redeployed -> the same pattern is now caught on retest.

5.4 Guarding against circularity (decoy features)

A real risk with synthetic data: the same person writes the data-generation logic and then asks an LLM to 'discover' the rules that produced it. Without a check, 'self-evolving' could quietly demo as 'converges back to code we already wrote,' which would not survive a sharp question about how this generalizes to real, messy data with confounders that weren't deliberately injected.

Two concrete guards, not just an acknowledgment:

- **Generator isolation** — as noted in Section 4.1, the Generator never sees the generation script, model weights, or phase/drift labels — only the schema and domain framing.
- **Decoy features** — a small number of columns are added to the dataset that correlate with *other* order attributes but have **no true causal relationship to RTO risk** in the generation logic (e.g. a cosmetic 'device model name' field, or 'app theme selected'). If the evolved system ends up assigning real fitness weight to a decoy, that is a direct, demonstrable signal of overfitting to spurious structure rather than genuine pattern discovery — and is reported honestly in the results if it happens, rather than hidden.

- **Blinded column naming (the stronger test)** — decoys alone only prove the Generator isn't copying the generation code; they don't prove it isn't just copying the human's feature-naming intuition, since well-named columns like 'pincode_rto_rate' or 'device_id_reuse_count' already hand a domain expert's causal hypothesis to the model before evolution starts. Fix: a second run is done where the causal columns are renamed generically (col_14, col_22, ...) with no semantic hint, alongside the same decoys. If the Generator still converges on the real signal-bearing columns without the naming hint, that is real evidence of discovery rather than intuition-following, and is a stronger, more specific claim than the decoy check alone.

This turns 'how do we know it's not just reverse-engineering your code' from a rhetorical question into something the video can actually show evidence against: decoys stayed unused, real signals (pincode history, device reuse, promo stacking) got picked up.

6. Production Guardrails

- **Low-confidence routing** — verdicts below a calibrated confidence threshold go to a Human Review Queue instead of an automatic block/approve, so the system never silently blocks a real customer on a marginal call.
- **Regression Suite** — every new generation must be re-validated against previously-solved cases before promotion, preventing 'catastrophic forgetting' where a fix for a new pattern breaks something that used to work.
- **Drift Detection** — a rolling-window check on both data drift (incoming feature distributions shifting) and performance drift (live precision/recall dropping), which is what triggers re-entry into the offline evolution phase. This is event-driven, not just running on a fixed timer.
- **Promotion / Rollback** — a new version only stays live if it matches or beats the previous version's live metrics; if it regresses, the system automatically reverts to the last known-good version.
- **Defense-only Audit** — an explicit checkpoint before any version is promoted, with a concrete two-step mechanism rather than a vague 'confirmed safe': (1) every hypothesis's plain-English rationale is passed through a second, separate LLM call prompted specifically to flag content that reads as evasion-instructional (e.g. 'how to structure an order to avoid detection') rather than detection-descriptive; (2) a simple keyword/pattern check over the rule code and rationale text as a fast first-pass filter before the LLM check runs. Given this maps directly to the track's literal disqualification clause, having a named mechanism here (not just 'we checked') is treated as non-optional, on the same footing as Gates 1 and 2.
- **Live Logging** — every prediction is logged with input, score, verdict, and eventual outcome once known, closing the feedback loop that feeds the Drift Detector.

6.1 Spike Monitor (fraud-spike detection, same loss class)

Default status: designed, not built by default. This component was proposed after the scope-protection plan (Section 9.1) was already written, which is itself a signal worth noting — the instinct to keep adding depth was fighting the instinct to protect the core loop. It remains good, defensible depth on the same loss class (see justification below), but given it would land on Sep 2, the same day as the Drift Detector, the default plan is to **describe it in the README as 'designed but descoped for time'** rather than build it — this captures nearly

the same credit for the idea without the calendar risk. Only build it if Sep 1 finishes ahead of schedule.

A second, distinct monitor alongside the Drift Detector, added to extend depth on the same COD/RTO loss class rather than to introduce a second one. The Drift Detector needs ground-truth outcomes, which lag by days (an order's true RTO/delivered status is only known after fulfillment plays out). The Spike Monitor instead watches the **raw flag-rate signal directly** — no ground truth needed — so it can catch a coordinated attack (bot-driven order flooding, a fraud ring hitting the site in a burst) in near-real-time instead of days later.

- **Signal** — rolling percentage of orders scored high-risk per time bucket (e.g., per hour).
- **Method** — statistical process control (z-score against a trailing baseline, or a CUSUM control chart); no model training required, easy to explain and defend to a panel.
- **On trigger** — raises an alert, logs the spike window for review, and can feed that window's orders into a targeted Generator round ('propose hypotheses informed specifically by this spike cluster') — reusing the existing evolution loop rather than building a separate system.
- **Reuses** — the same Live Logging stream already required for the Drift Detector; incremental build effort, not a new subsystem.

6.2 Human Review Queue — how it's counted in reported metrics

Left undefined, this is an easy gap for a panelist to find: orders routed to human review are the system's hardest cases by construction, so quietly excluding them from precision/recall would flatter the final numbers without anyone having to lie. This is decided explicitly, not left ambiguous:

- Held-out test metrics are reported **including** review-routed orders as a distinct third outcome class, not silently dropped — labeled as 'auto-decided' vs. 'routed to review' in the results table.
- Precision/recall on auto-decided orders is reported as the primary number (since that's what the automated system is actually responsible for), **alongside** the review-routed volume and its own outcome breakdown, so a panelist can see both numbers rather than one number that hides the hard cases.
- This is decided and documented before final held-out numbers are computed — not adjusted after seeing results, which would itself be a second, subtler form of the same honesty problem.

7. How the System Actually 'Understands' and Self-Evolves

This is not a vague claim — the mechanism is structured trial-and-error over real historical outcomes, in five concrete steps repeated every round:

1. Propose — Generator writes a concrete, testable rule or feature as executable code — not a feeling, a real condition an Evaluator can run.

2. Test against ground truth — Evaluator runs it on the validation split, where the true outcome (delivered vs. RTO) is already known, producing real precision/recall/cost numbers.

3. Show concrete failures — The Reflector isn't told 'recall is low' — it's shown specific missed or wrongly-flagged orders, with their actual feature values.

4. Diagnose and mutate — The Reflector reasons about why those specific cases were missed (e.g. 'this only checks pincode, but this case was device-reuse fraud with a safe pincode') and proposes a targeted next hypothesis.

5. Select, link, prune — The Selector keeps what's working, the Linker records how new and old hypotheses relate (overlap, contradiction, merge, refinement), and stale hypotheses are pruned — so the system's 'knowledge' compounds across rounds instead of restarting blind each time.

Critically, the 'intelligence' here is bounded and specific: a defined hypothesis space, a human-designed feature set, LLM-guided mutation reasoning over concrete evidence, and fitness verified by real code execution — not an autonomous system that independently understands fraud the way a human analyst would. This scope is stated explicitly in the demo rather than oversold, which is itself part of the defense-only, honest-metrics framing the track asks for.

8. Tech Stack

Layer	Tool
Hypothesis / feature generation, reflection	LLM API (Claude)
Data handling, evaluation	pandas, scikit-learn
Baseline trained model	LightGBM
Hypothesis relationship graph	networkx
Persistent memory (Notepad)	SQLite / JSON
Online serving	FastAPI
Dashboard / demo visualization	Streamlit
Drift detection	PSI / rolling-window precision-recall checks
Fraud-spike detection	z-score / CUSUM control chart on rolling flag-rate

Sandboxed rule execution	RestrictedPython / subprocess with timeout, no filesystem or network access
--------------------------	---

9. Build Plan (Aug 23 - Sep 5)

Dates	Focus
Aug 23-24	Synthetic RTO/COD dataset + train/validation/held-out split + injected drift scenario
Aug 25-26	Cost-weighted Evaluator + regression harness
Aug 27-28	Generator + Reflector agents (LLM-driven loop)
Aug 29	Selector + pruning logic + end-to-end loop test + snapshot frozen v1 (pre-drift-only) baseline
Aug 30-31	Knowledge graph hypothesis linking + lineage visualization
Sep 1	Low-confidence routing + defense-only audit pass + LightGBM baseline
Sep 2	Drift detection trigger + Spike Monitor (reuses Live Logging stream)
Sep 3	Promotion + rollback logic
Sep 4	Dashboard + final held-out test run + honest metrics report
Sep 5	Video, README, architecture doc, submit (buffer day, no new build work)

9.1 Scope protection — what to cut if behind schedule

Nine non-trivial components (Generator, Evaluator, Reflector, Selector, KG Linker, Notepad, three promotion gates, FastAPI serving, drift detector, promotion/rollback, Streamlit dashboard, LightGBM baseline, defense audit) is real scope for a solo build in under two weeks alongside coursework. "Failure Recovery" is a judged criterion, so partial credit exists for things breaking — but everything downstream of the core loop is decorative if the core loop itself isn't solid. Priority order if Aug 29 arrives behind schedule:

Priority	Component	If behind schedule
1 — never cut	Generator -> Evaluator -> Reflector -> Selector loop	This is the entire project. Protect at all costs.
1 — never cut	Train/validation/held-out split + honest metrics	This is the literal track bar.
2 — high value, cheap	Frozen v1 vs. evolved comparison (Sec 4.7)	The actual thesis proof. Cheap once the core loop works — a snapshot, not new code.
3 — cut second	Knowledge Graph Linker + lineage viz	Genuinely nice, not required by the track. First thing to simplify to plain mutation-only if short on time.
3 — cut second	Streamlit dashboard polish	A few clean charts beat a polished dashboard. Simplify, don't skip metrics.

4 — default descoped	Spike Monitor	Added after this plan was written — signal noted. Default: describe as 'designed, descoped for time' in README. Build only if Sep 1 finishes early.
4 — cut last resort	LightGBM baseline	Valuable but secondary comparison — narrate as 'considered, here's why not built' if truly out of time.

9.2 LLM API budget (operational risk, plan before Aug 27)

Not a design flaw, but an unaddressed operational risk: the Generator and Reflector both make LLM calls every round, the repair-retry loop (Section 4.1) can roughly double calls on rounds with failures, and this repeats across every evolution round run before Sep 5 — unattended, since the system is meant to run periodically without a human watching each call. Left unplanned, this risks hitting rate limits or burning through budget mid-build, discovered at the worst time.

- Estimate total calls before building: (Generator calls + Reflector calls) x hypotheses per round x rounds x retry-failure rate — a rough number should exist on paper before Aug 27, not be discovered live.
- Set an explicit per-round call cap (e.g. 3-5 new hypotheses per round, not open-ended) so cost scales predictably with round count.
- Batch or parallelize calls within a round where possible rather than issuing them serially.
- Have a stated fallback if approaching a limit mid-build: reduce hypotheses-per-round, disable the repair-retry loop temporarily, or pause evolution and resume rather than losing the run.

10. Anticipated Panel Questions

Q: What happens as hypotheses multiply?

A: Bounded top-k population; hypotheses unused/unselected for N rounds are pruned (archived, not deleted) — see Selector, Section 4.4.

Q: How do you know you're not overfitting across many evolution rounds?

A: Held-out test set is touched exactly once, after freezing — never during evolution.

Q: Isn't precision/recall unstable with a small fraud class?

A: k-fold / bootstrapped validation during evolution; confidence intervals reported on final test metrics, not single-point numbers.

Q: Your data is synthetic — how do we know this generalizes?

A: Stated plainly: no public dataset exists; distribution calibrated against published real-world RTO statistics.

Q: Isn't LLM-driven evolution slow/expensive for real-time use?

A: Evolution is offline/periodic; online inference uses only the frozen rule ensemble, no LLM call at scoring time.

Q: What stops it from chasing recall with excessive false positives?

A: Fitness is cost-weighted, not raw F1 — false positives are explicitly penalized in the fitness function itself.

Q: Could this teach someone how to evade RTO detection?

A: Hypotheses are framed as signals to watch for, never as exploit technique; a defense-only audit gate runs before every promotion.

Q: Why not just train a standard classifier?

A: A LightGBM baseline is trained and reported side-by-side on the same held-out set — the tradeoff (raw accuracy vs. interpretability and self-correction) is shown with real numbers, not assumed.

Q: You generated the data and labels — isn't the Generator just reverse-engineering your own code?

A: The Generator never sees the generation script or drift labels, only schema and domain framing; decoy features with no true causal link to RTO are included specifically to check for this — see Section 5.4.

Q: How did you calibrate the false-positive cost?

A: Stated explicitly as AOV x an assumed margin (a placeholder assumption pending real merchant data), reported alongside the sourced true-positive avoided-loss figure — see Section 4.2. Not left vague.

Q: Are human-review-routed orders counted in your final metrics?

A: Yes, reported as a distinct third outcome class alongside auto-decided precision/recall, not silently excluded — see Section 6.2.

Q: The decoys prove it's not copying your code — but is it just copying your feature-naming intuition?

A: A second run uses generically-renamed causal columns (col_14 instead of pincode_rto_rate) alongside the same decoys — if the Generator still finds the real signal without the naming hint, that's evidence of discovery, not intuition-following. See Section 5.4.

Q: Couldn't the frozen-v1 vs. evolved improvement just be from more optimization rounds, not drift adaptation?

A: Yes, that's a real confound, stated openly — see Section 4.7. The rigorous fix is a rounds-matched shadow control (same round budget, no drift exposure); treated as optional given the timeline, but the limitation is named rather than hidden.

Q: How is false-positive cost priced for the high-value orders you said were a key risk category?

A: Per-order value x an assumed margin, not a flat average AOV — priced consistently with the threat model rather than underpricing exactly the order type most likely to be flagged. See Section 4.2.

Q: What specifically confirms the defense-only audit — who or what is checking?

A: A keyword/pattern first pass, then a second separate LLM call prompted specifically to flag evasion-instructional content in each hypothesis's rationale, before promotion. See Section 6, Defense-only Audit.